

# AI SQL Assistant

## SQL Query Explanation and Optimization using IBM Granite 3.3 2B

---

### 1. Project Description

AI SQL Assistant is an intelligent database query analysis tool that uses the **IBM Granite 3.3 2B Instruct** model to analyze and interpret SQL queries. The system allows users to enter SQL queries and automatically generates a simple English explanation of the query along with optimization suggestions.

The main goal of this project is to help students, beginners, and database analysts understand SQL queries easily and improve query efficiency. The system also performs security validation to detect dangerous SQL operations such as DROP, DELETE, ALTER, and TRUNCATE to prevent malicious database operations.

The platform provides a user-friendly interface using **Gradio**, where users can input SQL queries, analyze them, and export reports of analyzed queries. The system keeps track of query history and allows users to download a report containing all previously analyzed queries.

This AI-powered assistant simplifies SQL learning and query analysis by converting technical SQL statements into understandable explanations while also offering optimization tips for better database performance.

### 2. Objectives of the Project

The main objectives of this project are:

- To develop an AI-powered SQL query explanation system
- To convert SQL queries into simple English explanations
- To provide optimization suggestions for SQL queries
- To detect dangerous SQL commands for security
- To provide an interactive chat interface for query analysis
- To allow exporting analyzed query history into a report

### 3. Core Technologies Used

The system is developed using the following technologies:

- **Python 3.x** – Main programming language
- **IBM Granite 3.3 2B Instruct** – AI model for SQL understanding
- **Transformers (Hugging Face library)** – Model loading and inference
- **Gradio** – Interactive user interface
- **PyTorch** – Deep learning framework
- **Google Colab** – Development and deployment environment

## 4. Pre-requisites

### Software Requirements

- Python 3.8 or above
- Internet connection
- Google account for Colab
- Web browser (Chrome, Firefox, Edge)

Required Python libraries:

transformers

gradio

torch

## 5. Knowledge Prerequisites

To understand and implement this project, the following knowledge is required:

- Basic Python programming
- SQL query fundamentals
- Understanding of databases and tables
- Basic knowledge of AI models
- Familiarity with web interfaces

## 6. Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: Minimum 2 GB free space
- Internet connection

Since the system runs on **Google Colab**, most processing is handled in the cloud.

---

## 7. Project Workflow

The project development process consists of several phases.

### Phase 1: Environment Setup

In this phase, the development environment is prepared.

Activities include:

- Creating a new notebook in **Google Colab**
- Installing required Python libraries
- Importing necessary modules
- Loading the AI model from Hugging Face

### Phase 2: AI Model Integration

The system loads the **IBM Granite 3.3 2B Instruct** model using the Transformers library.

The model is responsible for:

- Understanding SQL queries
- Generating human-readable explanations
- Assisting in query interpretation

### Phase 3: Security Validation

A security module is implemented to detect unsafe SQL queries.

The system checks for dangerous SQL keywords such as:

- DROP
- DELETE
- ALTER
- TRUNCATE

If any of these keywords are detected, the system blocks the query and displays an error message.

## Phase 4: SQL Explanation Engine

The SQL explanation module converts SQL queries into plain English descriptions.

Example:

Input SQL Query:

```
SELECT * FROM users WHERE age > 25
```

Output Explanation:

This query retrieves all records from the users table where the age of the user is greater than 25.

---

## Phase 5: SQL Optimization Suggestions

The system analyzes SQL queries and provides optimization tips.

Examples of suggestions include:

- Avoid using SELECT \*
- Add WHERE conditions to reduce data retrieval
- Use ORDER BY for sorting results

These suggestions help users write efficient database queries.

## Phase 6: User Interface Development

A web interface is created using **Gradio**.

The interface contains:

- SQL query input textbox
- Analyze SQL button
- Chat display area
- Export report button

This interface allows users to interact with the system easily.

## Phase 7: Export Feature

The system stores all analyzed queries in a history list.

Users can download a report containing:

- SQL queries entered
- Timestamp of analysis

The report is exported as a text file.

Example report file:

SQL:

```
SELECT * FROM users WHERE age > 25
```

Time:

2026-03-23

-----

## 8. System Architecture

The architecture of the AI SQL Assistant consists of the following components:

User Input

- Gradio Interface
- Security Validation Module
- AI Model Processing
- SQL Explanation Engine
- Optimization Engine
- Output Display
- Export Report

## 9. Functional Testing

The system is tested with different SQL queries.

### Test Case 1

Input:

```
SELECT * FROM users WHERE age > 25
```

Output:

Explanation of the query and optimization suggestions.

## Test Case 2

Input:

```
SELECT name,email FROM users
```

Output:

Explanation of query and suggestion to add WHERE condition.

## Test Case 3 (Security Test)

Input:

```
DROP TABLE users
```

Output:

Unsafe SQL query detected.

# 10. Deployment

The application is deployed using **Gradio**.

The system generates a public link when executed in **Google Colab**.

Example:

Running on public URL:  
<https://xxxx.gradio.live>

Users can access the application through this link.

# 11. Advantages of the System

- Helps beginners understand SQL queries
- Provides AI-powered explanations
- Detects dangerous SQL commands
- Suggests query optimization techniques
- Easy-to-use chat interface

## 12. Limitations

- Works mainly with simple SQL queries
- Requires internet connection for AI model
- Optimization suggestions are basic

## 13. Future Enhancements

The project can be improved by adding:

- Support for complex SQL queries
- JOIN query explanation
- Graphical query visualization
- Support for multiple database systems
- Advanced SQL performance analysis

## 14. Program

**# Install libraries**

**!pip install -q transformers gradio torch**

**import gradio as gr**

**from transformers import pipeline**

**from datetime import datetime**

**import re**

**from pathlib import Path**

```
# =====
```

```
# LOAD AI MODEL
```

```
# =====
```

```
pipe = pipeline(
```

```
    "text-generation",
```

```
    model="ibm-granite/granite-3.3-2b-instruct"
```

```
)
```

```
# =====
```

```
# SECURITY CHECK
```

```
# =====
```

```
DANGEROUS = ['DROP','DELETE','ALTER','TRUNCATE']
```

```
def check_security(sql):
```

```
    sql_upper = sql.upper()
```

```
    for word in DANGEROUS:
```

```
        if word in sql_upper:
```

```
            return False
```



```
return True
```

```
# =====
```

```
# SQL EXPLAINER
```

```
# =====
```

```
def explain_sql(sql):
```

```
    prompt = f"""
```

```
    Explain the following SQL query in simple English.
```

```
SQL:
```

```
{sql}
```

```
"""
```

```
    result = pipe(prompt, max_new_tokens=120)
```

```
    explanation = result[0]['generated_text']
```

```
    return explanation
```

```
# =====
```

```
# SQL OPTIMIZER
```

```
# =====
```

```
def optimize_sql(sql):
```

```
    suggestions = []
```

```
    if "SELECT *" in sql.upper():
```

```
        suggestions.append("Avoid SELECT *. Specify required  
columns.")
```

```
    if "WHERE" not in sql.upper():
```

```
        suggestions.append("Consider adding WHERE condition to reduce  
data.")
```

```
    if "ORDER BY" not in sql.upper():
```

```
        suggestions.append("Add ORDER BY for better sorting.")
```

```
    if not suggestions:
```

```
        suggestions.append("Query looks optimized.")
```

```
return "\n".join(suggestions)
```

```
# =====
```

```
# MAIN FUNCTION
```

```
# =====
```

```
history = []
```

```
def process_query(sql, chat_history):
```

```
    if not sql.strip():
```

```
        return chat_history
```

```
        chat_history.append((sql, "Processing..."))
```

```
    # security check
```

```
    if not check_security(sql):
```

```
        response = "✗ Unsafe SQL query detected."
```

```
    else:
```

```
explanation = explain_sql(sql)
```

```
optimization = optimize_sql(sql)
```

```
response = f" """
```

✓ SQL Explanation

```
{explanation}
```

⚡ Optimization Suggestions

```
{optimization}
```

```
"""
```

```
history.append({
```

```
    "sql":sql,
```

```
    "time":datetime.now().isoformat()
```

```
})
```

```
chat_history[-1]=(sql,response)
```

```
return chat_history
```

```
# =====
```

```
# EXPORT HISTORY
```

```
# =====
```

```
def export_txt():
```

```
    if not history:
```

```
        return "No history", None
```

```
    text="SQL Report\n\n"
```

```
    for h in history:
```

```
        text += f"    "
```

```
SQL:
```

```
{h['sql']}
```

```
Time:
```

```
{h['time']}
```

-----

"""

**filename="sql\_report.txt"**

**Path(filename).write\_text(text)**

**return "Downloaded",filename**

**# =====**

**# GRADIO INTERFACE**

**# =====**

**with gr.Blocks() as demo:**

**gr.Markdown("# 📄 AI SQL Assistant")**

**gr.Markdown("Explain and optimize SQL queries")**

**chatbot = gr.Chatbot(height=400)**

**query = gr.Textbox(**

```
label="Enter SQL Query",  
placeholder="Example: SELECT * FROM users WHERE age > 25"  
)
```

```
send = gr.Button("Analyze SQL")
```

```
send.click(process_query,inputs=[query,chatbot],outputs=[chatbot])  
query.submit(process_query,inputs=[query,chatbot],outputs=[chatbot])
```

```
gr.Markdown("## Export")
```

```
export_btn = gr.Button("Download Report")
```

```
file_out = gr.File()
```

```
export_btn.click(export_txt,outputs=[query,file_out])
```

```
demo.launch(share=True)
```

## Example Outputs

### 1 Example Query 1

#### Input

```
SELECT * FROM users WHERE age > 25;
```

## Output

### ✓ SQL Explanation

This SQL query retrieves all records from the **users** table where the **age** of the user is greater than 25.

### ✂ Optimization Suggestions

- Avoid using **\*\*SELECT\*\***. Specify only the required columns.
- Consider adding **ORDER BY** for sorted results.

## 2 Example Query 2

### Input

```
SELECT name, email FROM users WHERE country =  
'India';
```

### Output

### ✓ SQL Explanation

This query selects the **name** and **email** columns from the **users** table for users whose **country is India**.

### ✂ Optimization Suggestions

- Query looks optimized.
- You may add **ORDER BY name** for better sorting.

## 3 Example Query 3

### Input

```
SELECT product_name, amount FROM orders WHERE amount  
> 1000;
```

### Output

### ✓ SQL Explanation

This SQL query retrieves the **product name** and **amount** from the **orders** table where the order amount is **greater than 1000**.



## ⚡ Optimization Suggestions

- Add **ORDER BY amount DESC** to see highest orders first.

## 4 Example Query 4 (Security Test)

### Input

```
DROP TABLE users;
```

### Output

✗ Unsafe SQL query detected.

This query contains **dangerous SQL keywords** and cannot be processed.

## 5 Example Report Export

When you click **Download Report**, the program creates a file like:

SQL Report

SQL:

```
SELECT * FROM users WHERE age > 25
```

Time:

```
2026-03-23 14:05:20
```

-----

SQL:

```
SELECT name,email FROM users WHERE country='India'
```

Time:

```
2026-03-23 14:07:11
```

File generated:

```
sql_report.txt
```

## 15. Conclusion

The AI SQL Assistant project successfully demonstrates the use of artificial intelligence in database query analysis. By integrating the **IBM Granite 3.3 2B Instruct** model with an interactive interface built using **Gradio**, the system simplifies SQL query understanding and provides optimization guidance.

The platform bridges the gap between complex SQL queries and user comprehension by converting queries into simple English explanations. The addition of security validation ensures safe query analysis while preventing malicious operations.

This project highlights how AI can enhance database learning and improve query development for both beginners and professionals.